

# Compilation I

MU4IN501 – DLP : Développement d'un langage de programmation  
Master STL, Sorbonne Université

---

Antoine Miné

Cours 3

26 septembre 2023

Année 2023–2024



Développement d'un **langage** de programmation :  
**interprète** et **compilateur**.

Par étapes, avec ajout progressif de fonctionnalités.

- **ILP1** : langage de base

- constantes (entiers, flottants, booléens, chaînes),

- opérateurs (+, −, ×, ...),

- appels de primitives (print, ...),

- blocs locaux (let x = ... in ...),

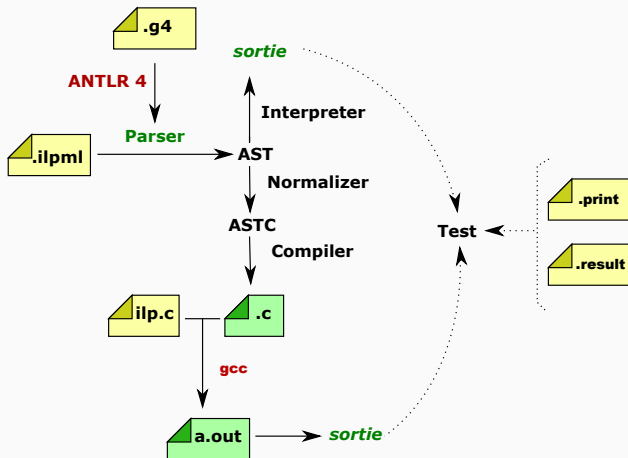
- alternatives (if ... then ... else)

- **ILP2** : ajout des boucles, affectations, fonctions globales

- **ILP3** : ajout des exceptions et fonctions de première classe

- **ILP4** : ajout des classes et des objets

# Schéma général d'ILP



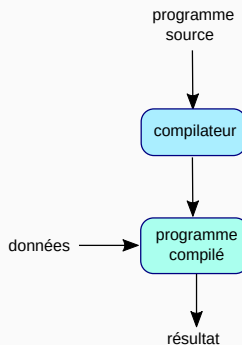
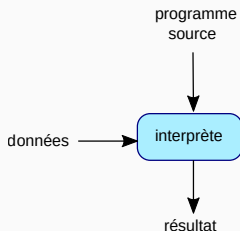
Après l'interprète du langage ILP1, nous voyons le **compilateur** sur deux séances.

## Cette semaine :

- généralités sur la compilation
- spécificités de la **compilation** d'**ILP1 vers le C**
- **représentation** des concepts ILP1 en C
  - valeurs
  - variables
  - instructions
- **bibliothèque d'exécution** en C
  - manipulation des valeurs (opérateurs)
  - primitives (affichage, ...)
  - initialisation et sortie du programme

La semaine prochaine : transformation d'AST et génération de code C.

# Compilation et interprétation



Avantages de l'**interprétation** :

- plus simple à construire
- plus facile à porter
- développement interactif (REPL)
- traits dynamiques (eval)

Avantages de la **compilation** :

- exécution plus rapide  
(spécialisation, optimisations)
- sources cachées  
(difficile à décompiler)
- exécutable autonome  
(bibliothèque d'exécution incluse)

# Modes d'exécution modernes des programmes

La compilation peut se faire :

- vers du **code natif** ou vers un **bytecode** portable
- **à l'avance** chez le développeur ou **à la volée** chez l'utilisateur

Exemples :

- **Pascal, Clang** : compilation vers du bytecode, recompilé vers du code natif
- **Python** : (CPython)  
compilation vers du bytecode  
interprété chez le client dans une machine virtuelle
- **Java** :  
compilation vers du bytecode, distribué au client  
recompilation *just-in-time* dans une machine virtuelle chez le client
- **JavaScript** : interprétation et compilation *just-in-time* chez le client  
(stratégies avancées de compilation : *tracing compiler*, etc.)
- **OCaml** : compilation en bytecode interprété, ou en assembleur natif
- **Haskell** : compilation en C, recompilé vers du code natif

Si un langage supporte plusieurs modes d'exécutions, un programme devrait donner le **même résultat** quel que soit le mode choisi !

# Aspects statiques, aspects dynamique

- **statique** : peut être déterminé avant l'exécution, sur le code source
- **dynamique** : ne peut être déterminé qu'au moment de l'exécution

## Statique :

- existence d'une variable
- type de la valeur d'une variable C
- appel de fonction C
- appel de méthode non-virtuelle C++
- existence d'un attribut Java
- hiérarchie de classes Java

## Dynamique :

- valeur d'une variable
- type de la valeur d'une variable Python
- appel de méthode Java
- appel de méthode virtuelle C++
- existence d'un attribut Python
- hiérarchie de classes Python

L'aspect statique ou dynamique d'un trait de langage **dépend du langage** !

Le compilateur :

- résout tous les **aspects statiques** du langage, par **analyse statique** du source  
(ensemble des variables, etc.)
- **génère le code** qui résoudra les **aspects dynamiques** à l'exécution

# Langages statiques, langages dynamiques

## Langages statiques : C, Java, OCaml

- plus d'aspects statiques à exploiter par le compilateur
- plus facile à optimiser (tables de méthodes pré-calculées, ...)
- plus faciles pour raisonner (par exemple, par **typage**)

## Cas particulier du C : un **assembleur portable**

- bibliothèque d'exécution optionnelle
- correspondance directe avec la machine (valeurs, opérations)
- opérations non sûres, typage faible

(comparer par exemple avec Java, un autre langage très statique)

## Langages dynamiques : JavaScript, Python, Scheme, **ILP**

- plus de décisions repoussées à l'exécution (typage)
- algorithmique complexe à l'exécution (surcharge et MRO en Python)
- **eval** : nécessite l'interprète complet dans la bibliothèque d'exécution !

L'analyse statique de langages dynamiques est possible, au prix de gros efforts !



Notions étudiées :

- représentation **bas-niveau** (C) de concepts **haut-niveau** (ILP)
- conversion et interopérabilité entre langages très différents !
- aspects **statiques** et aspects **dynamiques** en ILP

Le langage ILP, et son compilateur, seront **enrichis** en ILP3–4 avec des constructions de haut niveau absentes en C.

(exceptions, fonctions de premières classes, objets)

Ce dont on ne parle pas en DLP :

Les aspects **bas-niveau** de la compilation.

Ils sont encapsulés dans le compilateur C vers binaire.

représentation binaire des valeurs, allocation de registre, pile, sélection d'instruction, etc.

⇒ voir un cours de compilation classique

## Le compilateur ILP

---

## ILP : langage dynamique de haut niveau

- valeurs de type entier, booléen, flottant, chaîne
- variables (locales) non typées
- tout est expression et retourne une valeur (éventuellement false)
- structures de contrôle : alternative, séquence, bloc
- primitives : print, newline, to\_string, type\_of
- opérateurs : +, -, etc. polymorphes
- gestion automatique de la mémoire

# Caractéristiques d'ILP : comparaison avec le C

- **ILP :** valeurs de type entier, booléen, flottant, chaîne  
**C :** entiers, flottants, pointeurs
- **ILP :** variables (locales) non typées  
**C :** variables typées, avec représentation spécifique des valeurs du type
- **ILP :** tout est expression et retourne une valeur (éventuellement false)  
**C :** distinction entre instructions (sans valeur) et expressions
- **ILP :** structures de contrôle : alternative, séquence, bloc  
**C :** constructions similaires
- **ILP :** primitives print, newline  
**C :** bibliothèque C standard (optionnelle !)
- **ILP :** opérateurs +, -, etc. polymorphes  
**C :** opérateurs spécifiques à chaque type
- **ILP :** gestion automatique de la mémoire  
**C :** gestion manuelle de la mémoire (malloc, free)

Comment gommer ces différences ?

# Correspondance entre ILP et C

- **types ILP**  
⇒ **type C générique** `ILP_Object` pouvant contenir toute valeur ILP
- **variable ILP**  
⇒ **variable C** (de type `ILP_Object`)
- **séquence ILP**  
⇒ **séquence C**
- **bloc ILP**  
⇒ **bloc C**, avec portée locale des variables (règles de portée identiques)
- **if then else ILP**  
⇒ **if else C**
- **expression ILP** retournant une valeur  
⇒ compilée en un **bloc d'instructions C**  
et affectation de la valeur de retour dans une **variable temporaire**
- **programme ILP** retournant une valeur  
⇒ fonction d'entrée `ilp_program` retournant une valeur

Contrairement au C, ILP nécessite :

- une représentation **uniforme** des données
- des **informations de type dynamique** (à l'exécution)  
⇒ chaque valeur connaît son type
- une **bibliothèque d'exécution lourde**
  - gestion des **valeurs**
  - gestion des **primitives**
  - gestion automatique de la **mémoire**

### Note importante

Contrairement au Java, le C est difficilement extensible.

⇒ la bibliothèque d'exécution ILP1 contient en fait tout ILP1 à ILP4...

nous laisserons de côté pour l'instant ce qui touche aux classes, exceptions et clôtures  
en TME, une extension aura **le droit de recopier** les fichiers C pour les enrichir

## Exemples de code généré 1/4 : constante

source ILP

34

code C compilé

```
ILP_Object ilp_program()  
{  
    return ILP_Integer2ILP (34);  
}
```

- point d'entrée généré : `ilp_program`

le code de `main` est indépendant du programme ILP, et appelle la fonction `ilp_program()` générée  
voir fin du cours

- `ilp_program` retourne une valeur : la valeur ILP du programme
- une constante entière apparaît directement dans le code généré : 34
- mais une valeur C **doit être encapsulée** dans une valeur ILP : `ILP_Integer2ILP`  
stockée dans un `ILP_Object`

## Exemples de code généré 2/4 : expressions

source ILP

34 + 43

code C compilé

```
ILP_Object ilp_program() {  
    {  
        ILP_Object ilptmp2609;  
        ILP_Object ilptmp2610;  
        ilptmp2609 = ILP_Integer2ILP (34);  
        ilptmp2610 = ILP_Integer2ILP (43);  
        return ILP_Plus (ilptmp2609, ilptmp2610);  
    }  
}
```

- chaque sous-expression ILP donne lieu à une **variable temporaire** `ilptmpXXXX`  
⇒ cela simplifie la génération de code
- opérateur `+` → **appel à la bibliothèque d'exécution** : `ILP_Plus`



## Exemples de code généré 3/4 : variables

source ILP

```
let x = 1
and y = 2 in
x + y
```

code C compilé

```
ILP_Object ilp_program() {
{
    ILP_Object iltmp2669;
    ILP_Object iltmp2670;
    iltmp2669 = ILP_Integer2ILP (1);
    iltmp2670 = ILP_Integer2ILP (2);
    {
        ILP_Object x1 = iltmp2669;
        ILP_Object y2 = iltmp2670;
        {
            ILP_Object iltmp2671;
            ILP_Object iltmp2672;
            iltmp2671 = x1;
            iltmp2672 = y2;
            return ILP_Plus (iltmp2671, iltmp2672);
        }
    }
}
```

- variable ILP → variable C (de même nom, mais également numérotée pour éviter les ambiguïtés)
- bloc `let` ILP → bloc `{ }` (règles de portée identiques en ILP et en C)
- beaucoup de temporaires, et des blocs imbriqués pour limiter leur portée!  
nous pourrions optimiser, mais à quoi bon ? le compilateur C le fera pour nous !

## Exemples de code généré 4/4 : conditionnelles et primitives

source ILP

```
print(  
  if true then 1  
  else "toto"  
)
```

code C compilé

```
ILP_Object ilp_program() {  
  {  
    ILP_Object ilptmp96;  
    {  
      ILP_Object ilptmp97;  
      ilptmp97 = ILP_TRUE;  
      if (ILP_isEquivalentToTrue (ilptmp97))  
      {  
        ilptmp96 = ILP_Integer2ILP (1);  
      }  
      else  
      {  
        ilptmp96 = ILP_String2ILP ("toto");  
      }  
    }  
    return ILP_print (ilptmp96);  
  }  
}
```

- primitive ILP → appel à la bibliothèque d'exécution en C : `ILP_print`
- `if then else` ILP → `if { } else { }` en C
- valeur de vérité ILP : fonction `ILP_isEquivalentToTrue` en C
- **extraction du calcul de l'argument** de `print` dans un temporaire  
`print(e) → ILP_Object tmp = e; print(tmp)`

Sources spécifiques au compilateur :

- dans `Java/src/`
  - `com.paracamplus.ilp1.compiler`  
point d'entrée, définition des environnements, visiteur de génération de code C
  - `com.paracamplus.ilp1.compiler.interfaces`  
interfaces d'ASTC (version alternative de l'AST utilisée par le compilateur)
  - `com.paracamplus.ilp1.compiler.ast`  
classes d'ASTC
  - `com.paracamplus.ilp1.compiler.normalizer`  
conversion d'AST en ASTC
  - `com.paracamplus.ilp1.compiler.optimizer`  
transformation d'ASTC (identité pour l'instant, mais peut être utilisée pour l'optimisation)
  - `com.paracamplus.ilp1.compiler.test`  
tests unitaires JUnit 4
- dans `C/`
  - `ilp.[ch]`  
bibliothèque d'exécution pour ILP1 à ILP4
  - `gc-8.2.2` (mise à jour récente depuis 7.2!)  
gestion automatique de la mémoire : ramasse-miettes Boehm GC

## Représentation des valeurs ILP en C

---

Valeur ILP = bloc alloué sur le tas (`malloc`).

C'est une structure C contenant des champs :

- une **en-tête** `_class` indiquant le **type** de la valeur ILP  
entier, flottant, chaîne, etc.
- un **contenu** `_contents`
  - représentation dépendant du type de valeur ILP  
utilisation d'un type `union` en C pour énumérer tous les cas
  - taille fixe (entiers) ou variable (chaînes)

C'est une forme d'**union discriminée** (cf. type somme en OCaml).

C'est aussi un modèle de programmation objet avec classes.

cf. Smalltalk, Python  $\implies$  ce sera utile en ILP4...

## Note :

par soucis d'homogénéité, un type est aussi une valeur  
et a donc un type !

e.g. : entier  $\rightarrow$  type des entiers  $\rightarrow$  type des types  
bases de la réflexion en Java ou Smalltalk

```
typedef struct ILP_Object {
    struct ILP_Class* _class; // classe du type

    union {
        unsigned char asBoolean; // booléen

        int asInteger; // ou entier

        double asFloat; // ou flottant

        struct asString { // ou chaîne
            int _size;
            char asCharacter[1]; // taille réelle = _size
        } asString;

        struct asClass { // ou classe
            struct ILP_Class* super; // type parent
            char* name;
            int fields_count;
            struct ILP_Field* last_field; // liste chaînée d'attributs
            int methods_count;
            ILP_general_function method[1]; // taille réelle = methods_count
        } asClass;
        ...
    } _content; // contenu
} *ILP_Object;
```

```
typedef struct ILP_Class {
    struct ILP_Class* _class;
    union {
        struct asClass_ {
            struct ILP_Class*   super;
            char*                name;
            int                  fields_count;
            struct ILP_Field*    last_field;
            int                  methods_count;
            ILP_general_function method[2];
        } asClass;
    } _content;
} *ILP_Class;
```

Un type a :

- un type `_class`  $\mapsto$  `ILP_object_Class_class` (le type des types !)
- un nom `name`
- deux « méthodes » (`print` et `classOf`)

Pour l'instant, un objet `ILP_Class` est prédéfini pour chaque type de valeur ILP  
En ILP4, nous pourrions définir des nouveaux types : les classes du programme

Note : compatibilité entre valeurs génériques `ILP_Object` et classes `ILP_Class`

`ILP_Object._content.asClass`  $\equiv$  `ILP_Class._content.asClass`

## Exemples de types prédéfinis

ilp.c : type entier

```
struct ILP_Class ILP_object_Integer_class = {
    &ILP_object_Class_class,           // _class
    { { &ILP_object_Object_class,     // super
        "Integer",                     // name
        0,                             // fields_count
        NULL,                          // last_fields
        2,                             // methods_count
        { ILP_print,                  // method[]
          ILP_classOf } } }
};
```

ilp.c : type booléen

```
struct ILP_Class ILP_object_Boolean_class = {
    &ILP_object_Class_class,           // _class
    { { &ILP_object_Object_class,     // super
        "Boolean",                     // name
        0,                             // fields_count
        NULL,                          // last_fields
        2,                             // methods_count
        { ILP_print,                  // method[]
          ILP_classOf } } }
};
```

Un objet de type `ILP_Class` prédéfini pour chaque type ILP dans `ilp.c`.



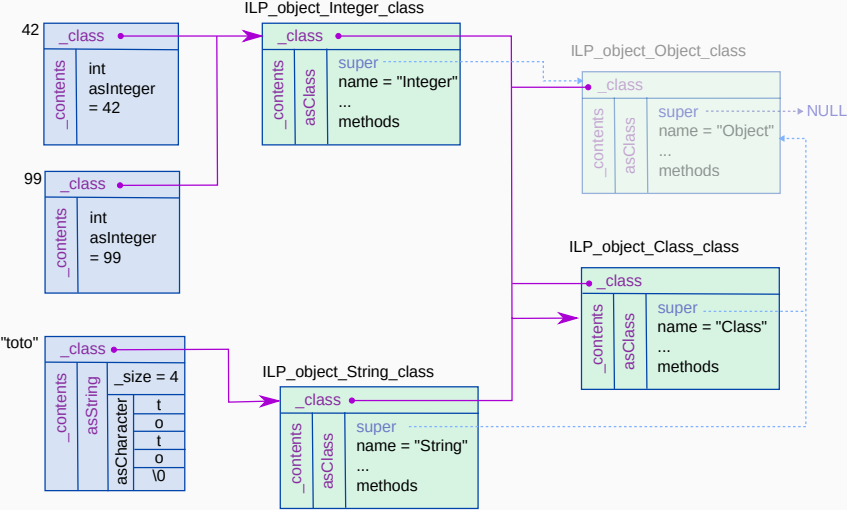
# Type des types (méta-classe)

ilp.c : type classe

```
struct ILP_Class ILP_object_Class_class = {
    &ILP_object_Class_class,      // _class
    { { &ILP_object_Object_class, // super
        "Class",                  // name
        1,                        // fields_count
        &ILP_object_super_field,  // last_fields
        2,                        // methods_count
        { ILP_print,              // method[]
            ILP_classOf } } }
};
```

ilp.c : type objet générique

```
struct ILP_Class ILP_object_Object_class = {
    &ILP_object_Class_class,      // _class
    { { NULL,                     // super
        "Object",                 // name
        0,                        // fields_count
        NULL,                     // last_fields
        2,                        // methods_count
        { ILP_print,              // method[]
            ILP_classOf } } }
};
```



## Bibliothèque d'exécution C

---

La **bibliothèque d'exécution** offre des services bas-niveau pour :

- **créer** des valeurs ILP d'un type donné, à partir d'une valeur C
- **déterminer** le type d'une valeur ILP
- **extraire** la valeur au type C natif (**int**, etc.)
- **gérer les erreurs** proprement (erreur de type, erreur numérique, etc.)

(sous forme de macros et de fonctions C)

et des services de plus haut niveau pour :

- implanter les **opérateurs** ILP
- implanter les **primitives** ILP

# Allocation de valeurs : appel bas niveau

ilp.c

```
ILP_Object ILP_malloc (int size, ILP_Class class)
{
    ILP_Object result = ILP_MALLOC(size);
    if ( result == NULL ) {
        return ILP_die("Memory exhaustion");
    }
    result->_class = class;
    return result;
}
```

Un objet est construit en précisant :

- son type : `class`
- la taille réelle à allouer : `size` (dépend du type et parfois de la valeur)

L'appelant devra renseigner le champ `_content` adéquat après construction.

(attention, ni le C ni `ILP_malloc` n'initialise ce champ !)

Des versions plus haut niveau pour des types précis existent. . . (transparents suivants)

`ILP_MALLOC` est `GC_malloc`, le *malloc* de la mémoire gérée par Bohem GC.

# Les entiers : création et test de type

ilp.h

```
#define ILP_AllocateInteger() \  
    ILP_malloc(sizeof(struct ILP_Object), &ILP_object_Integer_class)  
  
#define ILP_isInteger(o) \  
    ((o)->_class == &ILP_object_Integer_class)  
  
#define ILP_CheckIfInteger(o) \  
    if ( ! ILP_isInteger(o) ) { \  
        ILP_domain_error("Not an integer", o); \  
    };  
  
#define ILP_Integer2ILP(i) \  
    ILP_make_integer(i)
```

ilp.c

```
ILP_Object ILP_make_integer (int d)  
{  
    ILP_Object result = ILP_AllocateInteger();  
    result->_content.asInteger = d;  
    return result;  
}
```

Chaque `ILP_Class` représentant un type est un singleton.

Un test de type est réduit à tester `_class` avec `==` vis à vis du `ILP_Class` des entiers !

## Les booléens : création

ilp.h

```
enum ILP_BOOLEAN_VALUE {  
    ILP_BOOLEAN_FALSE_VALUE = 0,  
    ILP_BOOLEAN_TRUE_VALUE  = 1  
}  
  
#define ILP_TRUE  (&ILP_object_true)  
#define ILP_FALSE (&ILP_object_false)
```

ilp.c

```
struct ILP_Object ILP_object_true = {  
    &ILP_object_Boolean_class,  
    { ILP_BOOLEAN_TRUE_VALUE }  
}  
  
struct ILP_Object ILP_object_false = {  
    &ILP_object_Boolean_class,  
    { ILP_BOOLEAN_FALSE_VALUE }  
}  
  
ILP_Object ILP_make_boolean (int b)  
{  
    if ( b ) return ILP_TRUE;  
    else     return ILP_FALSE;  
}
```

Deux constantes statiques prédéfinies : **true** (ILP\_TRUE) et **false** (ILP\_FALSE).

Créer un booléen retourne toujours une des constantes préallouées  
sans allouer de nouvelle valeur (plus efficace en temps et en mémoire).

# Les chaînes : création et test de type

ilp.h

```
#define ILP_AllocateString(length) \
    ILP_malloc(sizeof(struct ILP_Object) \
        + (sizeof(char) * (length)), &ILP_object_String_class)

#define ILP_isString(o) \
    ((o)->_class == &ILP_object_String_class)

#define ILP_CheckIfString(o) \
    if ( ! ILP_isString(o) ) { \
        ILP_domain_error("Not a string", o); \
    }

#define ILP_String2ILP(s) \
    ILP_make_string(s)
```

ilp.c

```
ILP_Object ILP_make_string (char *s)
{
    int size = strlen(s);
    ILP_Object result = ILP_AllocateString(size);
    result->_content.asString._size = size;
    memmove(result->_content.asString.asCharacter, s, size);
    return result;
}
```

La taille allouée dépend de la longueur de la chaîne...



ilp.c

```
ILP_Object ILP_domain_error (char *message, ILP_Object o)
{
    snprintf(ILP_the_exception._content.asException.message,
             ILP_EXCEPTION_BUFFER_LENGTH,
             "Domain error: %s\nCulprit: 0x%p\n",
             message, (void*) o);
    fprintf(stderr, "%s", ILP_the_exception._content.asException.message);
    ILP_the_exception._content.asException.culprit[0] = o;
    ILP_the_exception._content.asException.culprit[1] = NULL;
    return ILP_throw((ILP_Object) &ILP_the_exception);
}
```

La bibliothèque d'exécution ajoute au C une notion d'**exceptions**.

Les détails seront précisés dans le cours sur ILP3...

Si on se limite à ILP1 (exceptions non rattrapées) :

- **ILP\_domain\_error** crée un objet contenant un message d'erreur
- retourne le contrôle directement au **main** du programme en « sautant » la pile d'appels (**ILP\_throw**)
- **main** affiche le message d'erreur et quitte

## Exemples d'opérateur 1/4 : la comparaison <

```
#define ILP_LessThan(o1,o2) ILP_compare_less_than(o1,o2)

#define DefineOrderComparator(name,op) \
ILP_Object ILP_compare_##name (ILP_Object o1, ILP_Object o2) { \
    if ( ILP_isInteger(o1) ) { \
        if ( ILP_isInteger(o2) ) { \
            return ILP_make_boolean( \
                o1->_content.asInteger op o2->_content.asInteger); \
        } else if ( ILP_isFloat(o2) ) { \
            return ILP_make_boolean( \
                o1->_content.asInteger op o2->_content.asFloat); \
        } else return ILP_domain_error("Not a number", o2); \
    } else if ( ILP_isFloat(o1) ) { \
        // ... \
        // cas symétrique \
    } else if ( ILP_isString(o1) && ILP_isString(o2) ) { \
        return ILP_make_boolean( \
            strcmp( o1->_content.asString.asCharacter, \
                o2->_content.asString.asCharacter ) op 0); \
    } else return ILP_domain_error("Non comparable", o1); \
}

DefineOrderComparator(less_than, <)
```

5 cas à gérer : entier/entier, entier/flottant, flottant/entier, flottant/flottant, chaîne/chaîne.  
Plus les cas d'erreur...

⇒ utilisation de **macros** pour factoriser le code de <, <=, > et >=.

## Exemples d'opérateur 2/4 : la soustraction

```
#define DefineOperator(name,op) \
ILP_Object ILP_make_##name (ILP_Object o1, ILP_Object o2) { \
    if ( ILP_isInteger(o1) ) { \
        if ( ILP_isInteger(o2) ) { \
            ILP_Object result = ILP_AllocateInteger(); \
            result->_content.asInteger = \
                o1->_content.asInteger op o2->_content.asInteger; \
            return result; \
        } else if ( ILP_isFloat(o2) ) { \
            ILP_Object result = ILP_AllocateFloat(); \
            result->_content.asFloat = \
                o1->_content.asInteger op o2->_content.asFloat; \
            return result; \
        } else return ILP_domain_error("Not a number", o2); \
    } else if ( ILP_isFloat(o1) ) { \
        ... \
    } else return ILP_domain_error("Not a number", o1); \
} \

DefineOperator(subtraction, -)
```

4 cas à gérer, et le type de retour dépend du type des arguments :

- entier/entier → retourne un entier
- entier/flottant, flottant/entier, flottant/flottant → retourne un flottant

Factorisation du code d'opérateurs similaires `*`, `/` par une macro ; `+` est plus complexe.

## Exemples d'opérateur 3/4 : l'addition

```
ILP_Object ILP_make_addition (ILP_Object o1, ILP_Object o2)
{
    if ( ILP_isString(o1) || ILP_isString(o2) ) {
        ILP_Object v1 = ILP_isString(o1) ? o1 : ILP_to_string(o1);
        ILP_Object v2 = ILP_isString(o2) ? o2 : ILP_to_string(o2);
        return ILP_concatenate_strings(v1, v2);
    } else if ( ILP_isInteger(o1) ) {
        if ( ILP_isInteger(o2) ) {
            ILP_Object result = ILP_AllocateInteger();
            result->_content.asInteger =
                o1->_content.asInteger + o2->_content.asInteger;
            return result;
        } else if ( ILP_isFloat(o2) ) {
            ILP_Object result = ILP_AllocateFloat();
            result->_content.asFloat =
                o1->_content.asInteger + o2->_content.asFloat;
            return result;
        } else return ILP_domain_error("Not a number", o2);
    } else if ( ILP_isFloat(o1) ) {
        ...
    } else return ILP_domain_error("Not addable", o1);
}
```

Similaire à la soustraction, mais gère aussi le cas chaîne (avec conversion implicite si nécessaire).  
Cas unique, donc pas d'utilisation de macro.

## Exemples d'opérateur 4/4 : les opérations booléennes

```
#define ILP_isEquivalentToTrue(o) ((o) != ILP_FALSE)

#define ILP_isTrue(o) \
    (((o)->_class == &ILP_object_Boolean_class) && ((o)->_content.asBoolean))

ILP_Object ILP_and (ILP_Object o1, ILP_Object o2)
{
    if ( ILP_isEquivalentToTrue(o1) ) {
        if ( ILP_isEquivalentToTrue(o2) ) return o2;
    }
    return ILP_FALSE;
}

ILP_Object ILP_make_negation (ILP_Object o)
{
    ILP_CheckIfBoolean(o);
    if ( ILP_isTrue(o) ) return ILP_FALSE;
    else return ILP_TRUE;
}
```

- toute valeur non **false** est considérée comme vraie pour un test
- le « et booléen » retourne son deuxième argument, ou faux  
⇒ le résultat n'est pas forcément un booléen !
- le « non booléen » n'accepte que des arguments booléens

## Exemples de primitive 1/3 : l'affichage

ilp.c

```
ILP_Object ILP_print (ILP_Object self)
{
    if ( self->_class == &ILP_object_Integer_class ) {
        fprintf(stdout, "%d", self->_content.asInteger);
    } else if ( self->_class == &ILP_object_Float_class ) {
        fprintf(stdout, "%12.5g", self->_content.asFloat);
    } else if ( self->_class == &ILP_object_Boolean_class ) {
        fprintf(stdout, "%s", (ILP_isTrue(self) ? "true" : "false"));
    } else if ( self->_class == &ILP_object_String_class ) {
        fprintf(stdout, "%s", self->_content.asString.asCharacter);
    } else if ( self->_class == &ILP_object_Class_class ) {
        fprintf(stdout, "<Class:%s>", self->_content.asClass.name);
    } else
        ...
    return ILP_FALSE;
}
```

- affichage spécifique en fonction du type
- la primitive retourne toujours `false`
- `ILP_to_string` est similaire, mais retourne une chaîne ILP

## Exemples de primitive 2/3 : le type

ilp.c

```
ILP_Object ILP_type_of (ILP_Object self)
{
    return ILP_make_string( self->_class->_content.asClass.name );
}
```

- extrait le type `self->_class` de la valeur `self`
- extrait le nom `name` de ce type
- convertit la chaîne C en chaîne ILP avec `ILP_make_string`

## Exemples de primitive 3/3 : la constante $\pi$

ilp.h

```
#define ILP_PI_VALUE 3.1415926535
#define ILP_PI (ILP_pi())
```

ilp.c

```
ILP_Object ILP_pi ()
{
    static ILP_Object object_pi = NULL;
    if ( object_pi == NULL ) {
        object_pi = ILP_make_float(ILP_PI_VALUE);
    }
    return object_pi;
}
```

code généré pour le programme pi

```
ILP_Object ilp_program ()
{
    return ILP_PI;
}
```

- du côté ILP, la constante est vue comme une **référence de variable ILP\_PI**
- la bibliothèque d'exécution la transforme en **appel de fonction ILP\_pi()**
- utilisant un **cache** pour créer la valeur flottante  $\pi$  en ILP au plus une fois



code généré fixe

```
static ILP_Object ilp_caught_program () {
    struct ILP_catcher *current_catcher = ILP_current_catcher;
    struct ILP_catcher new_catcher;
    if (0 == setjmp (new_catcher._jmp_buf))
    {
        ILP_establish_catcher (&new_catcher);
        return ilp_program ();
    };
    return ILP_current_exception;
}

int main (int argc, char *argv[]) {
    ILP_START_GC;
    ILP_print (ilp_caught_program ());
    ILP_newline ();
    return EXIT_SUCCESS;
}
```

Code de `main` figé pour tous les programmes :

- démarrage du GC : `ILP_START_GC`
- gestionnaire d'exception *top-level* (voir cours sur les exceptions ILP)
- lancement du programme ILP proprement dit (code généré) : `ilp_program`
- affichage de la valeur retournée : `ILP_print`

## Extensions

---

Il est possible d'**étendre** le langage sans changer la syntaxe, en changeant uniquement la bibliothèque d'exécution :

- **enrichir** les **primitives** et les **opérateurs existants**  
e.g. : entier \* chaîne
- **ajouter** des **primitives**  
e.g. : primitive **sin**, entrées/sorties
- **ajouter** des **structures de données**  
et les primitives nécessaires pour les créer et les manipuler  
e.g. : `make_vector(10)`, `set_vector(v,0,"a")`, `get_vector(v,1)`  
e.g. : fichiers

Cf. TME 3

# Aspects statiques et dynamiques

Lors de l'ajout d'un trait de langage, essayez toujours de déterminer ce qui est **statique** et ce qui est **dynamique**.

Exemple : variables existantes et définies, cf. TME 10

— exists et defined —

```
function f(a) (  
  // a, g0, g1, g2 existent, pas b  
  // a, g0 sont définies, pas g1 ni g2  
  g2 = a  
  // g2 est défini  
);  
let b = 1 in  
g0 = b;  
f(g0);  
g1 = 12
```

Ajout de deux mots-clés :

- **exists(v)** rend vrai si la variable **v** existe  
(locale dans la portée, ou globale référencée n'importe où dans le programme)  
⇒ **statique**
- **defined(v)** rend vrai si la variable **v** a été initialisée  
(variable existante qui a été affectée au moins une fois durant l'exécution)  
⇒ **dynamique**

# Optimisation : simplification du code

## code original

```
let x = 2 * 10 in  
let y = x + 1 in  
print(y + 2)
```

## propagation de constantes

```
let x = 20 in  
let y = 21 in  
print(23)
```

## élimination

```
print(23)
```

Optimisation : amélioration de l'efficacité du code compilé par :

- **transformation** du code en un code plus efficace  
(transformation source → source, ou optimisation de la génération de code)
- **analyse statique** du code (à la compilation)  
pour découvrir des propriétés permettant plus de transformations

## Exemple : simplification des constantes

- détermination des variables constantes : **x, y**
- évaluation des expressions constantes :  $2 \times 10 \rightarrow 20$
- élimination des variables devenues inutiles : **x, y**

⇒ gain de place mémoire  
**plus de travail fait à la compilation, moins à l'exécution** (dynamique → statique)  
l'ordre des passes d'optimisation compte ! (propager avant d'éliminer, pour mieux éliminer)

# Optimisation : spécialisation de type

code générique

```
ILP_Object res;  
ILP_Object x1 = v;  
ILP_Object y2 = w;  
res = ILP_Times( x1, ILP_Plus( y2, ILP_Integer2ILP( 1 ) ) );
```

code spécialisé

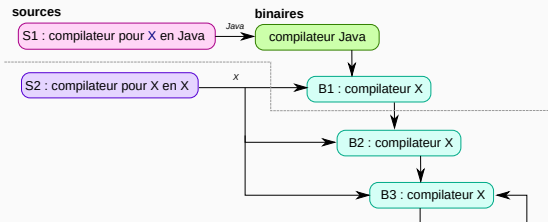
```
ILP_Object res;  
int x1 = v._contents.asInteger;  
int y2 = w._contents.asInteger;  
res = ILP_Integer2ILP( x1 * (y2 + 1) );
```

ILP est un langage **dynamiquement typé**

→ chaque variable, chaque calcul intermédiaire alloue un `ILP_Object`  
chaque opération effectuée un test de type (coûteux !)

Optimisation : **spécialisation de type** (particulièrement utile en JavaScript !)

- déterminer si une variable ou un opérateur sera toujours utilisé avec le même type, par **analyse statique**
- utiliser un **type C** et des **opérateurs natifs**, si possible  
⇒ gain de place mémoire et d'efficacité
- dans le doute, laisser le type générique



Développement classique d'un langage X :

- écriture d'un compilateur S1 **initial en Java ou C** (pour un sous-ensemble de X)  
→ compilateur binaire B1 non optimisant, non optimisé
- écriture d'un compilateur S2 **optimisant en X**, compilé avec B1  
**le but ultime d'un langage est de servir à écrire son propre compilateur !**  
→ compilateur binaire B2 optimisant, non optimisé
- recompilation de S2 avec B2  
→ **compilateur binaire B3 optimisant, optimisé**
- vérification : compiler S2 avec B3 redonne B3 !

Mais ILP se limite à la première étape : B1 (compilateur S1 pour ILP vers C écrit en Java)...

- le **compilateur** n'exécute pas, mais **génère du code** (en C pour ILP) dont l'**exécution** donnera le **résultat** escompté
- le **compilateur** résout les **aspects statiques** du programme et délègue la résolution des **aspects dynamiques** au **code généré**
- pour un langage **dynamique**, plus est fait **à l'exécution** :
  - **représentation** verbeuse des valeurs en mémoire (**ILP\_Object**)
  - **tests de type** à l'exécution (**ILP\_IsInteger**)
  - **sémantique des opérateurs** dépendante du type des valeurs (**ILP\_Plus**), avec éventuellement départ en erreur

ce choix a donc une **incidence sur l'efficacité** et **la sûreté**

- un langage **dynamique de haut niveau** nécessite une **bibliothèque d'exécution lourde** liée au code généré et distribuée avec tout exécutable

Dans ce cours, nous avons vu la bibliothèque d'exécution.

Il ne reste plus qu'à voir la transformation de l'AST en code C ! (prochain cours)